

COP290 - Design Practices

Assignment 3: Making Changes in LevelDB

LevelDB: Range Scan, Range Delete, and Manual Full Compaction

Course	COP290 - Design Practices
Assignment	3
Fenil Mayank Patel	2024CS10519
Megh Bhavesh Jesalpura	2024CS10844
Github	https://github.com/MeghJesalpura/cop290_a3.git

Contents

1	LevelDB Internals: Write Path, Read Path, and Compaction	3
1.1	The Write Path	3
1.2	The Read Path	4
1.3	The Compaction Mechanism	5
1.4	The Iterator Interface	6
2	Range Scan: Scan(start_key, end_key)	7
2.1	API Definition	7
2.2	Implementation	7
2.2.1	Why the iterator is the right abstraction	8
2.2.2	Correctness	8
3	Range Delete: DeleteRange(start_key, end_key)	9
3.1	API Definition	9
3.2	Implementation	9
3.2.1	Semantics	10
3.2.2	Multithreading behavior	10
3.2.3	Efficiency note	11
4	Manual Full Compaction: ForceFullCompaction()	11
4.1	API Definition	11
4.2	Compaction Statistics and Concurrency Fields	11
4.3	ForceFullCompaction Implementation	12
4.4	Deadlock Avoidance in TEST_CompactMemTable	14
4.5	Metric Collection in the Compaction Worker	15
5	Summary of File Changes	15
6	End-to-end Sanity Test	16

6.1	What the test does	16
6.2	Output	16
6.3	Reading the FFC numbers	17

1 LevelDB Internals: Write Path, Read Path, and Compaction

Before jumping into what we changed, it helps to understand how LevelDB works under the hood. The codebase was pretty large but the documentation was helpful!

1.1 The Write Path

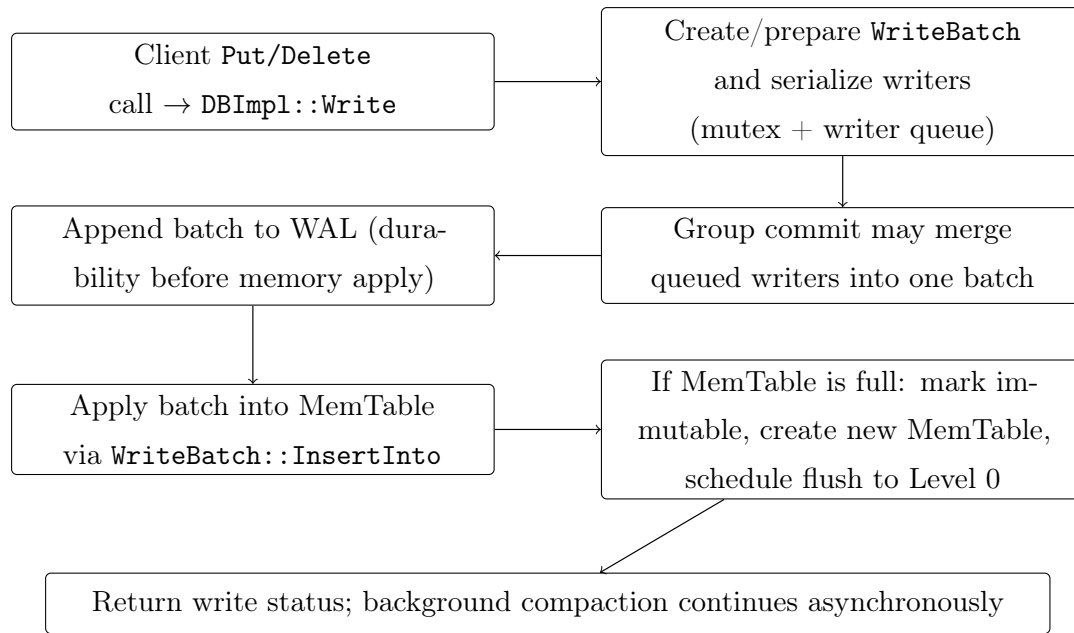
Every write - whether a `Put` or a `Delete` - goes through `DBImpl::Write` in `db/db_impl.cc`. The first thing that happens is the write gets packaged into a `WriteBatch`. Even single-key operations end up as a batch internally; see how `DB::Put` and `DB::Delete` are implemented in `db/db.cc` - they both construct a `WriteBatch` and call `Write`.

Inside `DBImpl::Write`, concurrent writers are serialized using a mutex and a writer queue. Only one writer leads at a time, and it may absorb other queued writers into a single grouped batch (the “group commit” optimization, visible in `db_impl.cc`).

Once the batch is ready, it goes through two things in sequence:

1. **Write-Ahead Log (WAL):** The batch is appended to the log file (`db/log_writer.cc`) before anything touches memory. This is what makes the database crash-safe.
2. **MemTable insertion:** The batch is applied to the in-memory `MemTable` via `WriteBatch::InsertInto` (`db/write_batch.cc`), which calls `MemTableInserter` - a handler that walks through each record and either calls `MemTable::Add` for puts or records a deletion tombstone.

The write flow can be summarized as:



WAL append happens before MemTable insertion, preserving crash safety.

The `MemTable` is a skip list (`db/skiplist.h`) sorted by internal key, which encodes the user key, a sequence number, and a value type. Sequence numbers monotonically increase, which is how MVCC is handled: newer writes shadow older ones with the same key.

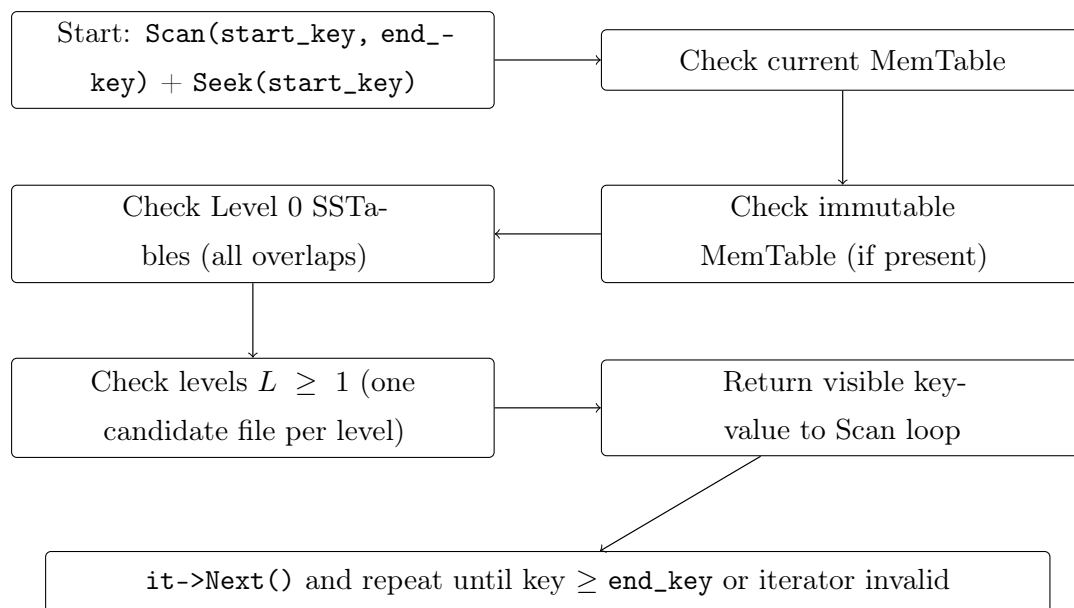
When the `MemTable` grows past a threshold (default 4 MB, set in `include/leveldb/options.h`), it is marked immutable and a new `MemTable` takes over. The immutable `MemTable` is then scheduled to be flushed to an `SSTable` at Level 0 in the background via `DBImpl::MaybeScheduleCompaction`.

1.2 The Read Path

Reads go through `DBImpl::Get` in `db/db_impl.cc`. The lookup order is:

1. **Current MemTable** - most recent writes land here.
2. **Immutable MemTable** - if it exists (i.e., a flush is in progress).
3. **SSTables, level by level** - using `Version::Get` in `db/version_set.cc`, which consults the current `Version` to find which `SSTable` files could contain the key.

For `Scan(start_key, end_key)`, the same read hierarchy is traversed through the merged iterator. The flow is:



If a key is not found at one layer, the iterator falls through to the next lower layer.

At Level 0, all files are checked because their key ranges can overlap. At Level 1 and below, the files are non-overlapping within a level, so a binary search narrows it to at most one file per level. The actual SSTable lookup (`table/table.cc`) checks the Bloom filter first, then binary-searches the data block index, and finally scans the appropriate block.

1.3 The Compaction Mechanism

Compaction is managed in `DBImpl::BackgroundCompaction` in `db/db_impl.cc`. There are two kinds:

Size-triggered compaction. When a level exceeds its size budget, `VersionSet::PickCompaction` selects files from that level along with any overlapping files from the next level. These are merged, sorted, and written as new SSTables at the next level.

Seek-triggered compaction. If a file is sought too many times without a hit (tracked in `allowed_seeks` per file), it gets scheduled for compaction regardless of size.

The pathway usually starts at Level 0. Fresh SSTables from MemTable flushes land here, and unlike deeper levels, Level 0 files are allowed to overlap in key range. Once there are too many Level 0 files (the classic threshold is around four files), reads start paying a merging penalty, so LevelDB prioritizes compaction from Level 0 to Level 1. This first step is intentionally aggressive because it is where write bursts can quickly translate into read amplification.

For a general level L , compaction picks one file from level- L and then pulls in *all* overlapping files from level- $(L + 1)$. A subtle but important detail is that even partial overlap is enough to include a full level- $(L + 1)$ file as input. So in practice, one small overlap can widen the compaction set. Level 0 is treated specially: because files can overlap each other there, the picker may include multiple Level 0 files in the same compaction if their key ranges intersect.

After inputs are chosen, the merged stream is written out as a sequence of new files at level- $(L + 1)$. LevelDB does not keep appending forever into one huge output file: it rolls over to a new output around the target file size (roughly 2 MB), and it can also roll over early if the current output key range would overlap too many files at level- $(L + 2)$ (the well-known “too wide overlap” guard, around 10 files). This keeps future compactions from becoming unnecessarily expensive.

Another nice design choice is that compaction does not keep hammering the same key region repeatedly. For each level, LevelDB remembers the end key of the previous compaction and picks the next file after that point, wrapping around the key space when needed. Over time this gives a round-robin style sweep across the level. During the merge itself, obsolete versions are dropped, and deletion tombstones are removed once there are no higher levels whose file ranges could still contain older values for that key.

The actual work happens in `DBImpl::DoCompactionWork`, which creates a merge iterator over all input files and streams through key-value pairs. During this scan, stale versions (older sequence numbers for the same key) and deletion tombstones that have propagated past the bottom level are dropped.

1.4 The Iterator Interface

LevelDB exposes a unified iterator over the full database state via `DBImpl::NewIterator`. Internally it creates a `MergingIterator` that combines the `MemTable` iterator, the immutable `MemTable` iterator, and individual `SSTable` iterators across all levels. This is the abstraction our `Scan` implementation builds on.

The core iterator construction path is shown below:

```
1 Iterator* DBImpl::NewIterator(const ReadOptions& options) {  
2     SequenceNumber latest_snapshot;  
3     uint32_t seed;  
4     Iterator* iter = NewInternalIterator(options, &latest_snapshot, &
```

```

        seed);
5   return NewDBIterator(this, user_comparator(), iter,
6       (options.snapshot != nullptr
7       ? static_cast<const SnapshotImpl*>(options.snapshot)
8       ->sequence_number()
9       : latest_snapshot),
10      seed);
11 }

```

Listing 1: DBImpl::NewIterator in db/db_impl.cc

2 Range Scan: Scan(start_key, end_key)

2.1 API Definition

We added the following to include/leveldb/db.h:

```

1 virtual Status Scan(
2     const ReadOptions&,
3     const Slice& start_key,
4     const Slice& end_key,
5     std::vector<std::pair<std::string, std::string>>* result) = 0;

```

Listing 2: Scan API declaration in include/leveldb/db.h

We also added `#include <vector>` at `db.h` since the signature requires `std::vector`. The corresponding override is declared in `db/db_impl.h`.

The half-open interval `[start_key, end_key)` matches the convention used by LevelDB's own comparators and makes it natural to compose adjacent scans without overlap or gap.

2.2 Implementation

The implementation lives in `db/db_impl.cc`. The key idea: grab the database's merged iterator, seek to `start_key`, then walk forward until the current key hits or passes `end_key`, collecting everything along the way.

```

1 Status DBImpl::Scan(const ReadOptions& read_opt, const Slice&
2     start_key,
3     const Slice& end_key,

```

```
3         std::vector<std::pair<std::string, std::string>>*  
           result) {  
4     leveldb::Iterator* it = this->NewIterator(read_opt);  
5     for (it->Seek(start_key);  
6         it->Valid() && user_comparator()->Compare(it->key(), end_key) <  
           0;  
7         it->Next()) {  
8         result->push_back({it->key().ToString(), it->value().ToString()});  
9     }  
10    Status s = it->status();  
11    delete it;  
12    return s;  
13 }
```

Listing 3: Scan implementation in db/db_impl.cc

2.2.1 Why the iterator is the right abstraction

The iterator returned by `DBImpl::NewIterator` internally creates a `MergingIterator` over the `MemTable`, immutable `MemTable`, and all `SSTables` across all levels. It respects the sequence number snapshot in the `ReadOptions`, giving us a consistent view of the database. We don't have to think about merging data from different levels ourselves.

2.2.2 Correctness

Each key yielded by the iterator reflects the latest visible version of that key, exactly as `Get` would return it. Deletion tombstones are never surfaced through the iterator - they are hidden by the internal key comparison logic. The result vector therefore only contains live key-value pairs.

If no keys fall in the given range, the loop does not execute and the function returns an empty vector with `Status::OK()`. If the iterator encounters an I/O error partway through, `it->status()` will be non-OK and we return that error after cleaning up the iterator.

3 Range Delete: DeleteRange(start_key, end_key)

3.1 API Definition

Added to include/leveldb/db.h:

```

1 virtual Status DeleteRange(
2     const WriteOptions&,
3     const Slice& start_key,
4     const Slice& end_key) = 0;

```

Listing 4: DeleteRange API declaration in include/leveldb/db.h

The override is declared in db/db_impl.h.

3.2 Implementation

The implementation is in db/db_impl.cc. The approach is still to scan the key range, build a WriteBatch, and commit it atomically. However, the latest two pushes added explicit write-stall handling so DeleteRange executes as an exclusive write phase.

```

1 Status DBImpl::DeleteRange(const WriteOptions& write_opt,
2                             const Slice& start_key, const Slice&
3                                 end_key) {
4     {
5         MutexLock l(&mutex_);
6         while (writes_blocked_.load(std::memory_order_acquire)) {
7             background_work_finished_signal_.Wait();
8         }
9         // Gate new writers BEFORE draining; otherwise the drain releases
10        mutex_
11        // in Wait() and fresh writers can keep enqueueing, starving us.
12        writes_blocked_.store(true, std::memory_order_release);
13        while (!writers_.empty()) background_work_finished_signal_.Wait();
14    }
15
16    leveldb::Iterator* it = this->NewIterator(leveldb::ReadOptions());
17    leveldb::WriteBatch batch;
18    for (it->Seek(start_key);
19         it->Valid() && user_comparator()->Compare(it->key(), end_key) <
20             0;
21         it->Next()) {
22        batch.Delete(it->key());
23    }
24    batch.Commit(write_opt);
25    return Status::OK();
26 }

```

```
20 }
21 Status s = it->status();
22 if (s.ok()) {
23     s = this->WriteInternal(write_opt, &batch, true);
24 }
25 delete it;
26
27 {
28     MutexLock l(&mutex_);
29     writes_blocked_.store(false, std::memory_order_release);
30     background_work_finished_signal_.SignalAll();
31 }
32
33 return s;
34 }
```

Listing 5: DeleteRange implementation in db/db_impl.cc

3.2.1 Semantics

This follows the same tombstone-based deletion model as single-key `Delete`. Each key in the range gets a deletion marker written into the MemTable. The underlying SSTable data is not touched immediately - it will be cleaned up during compaction when tombstones are merged with the corresponding data files.

The tombstones are written atomically as a single `WriteBatch`, meaning either all of them land or none do (the WAL guarantees this across crashes).

3.2.2 Multithreading behavior

The updated implementation explicitly coordinates with concurrent writers:

1. Under `mutex_`, it waits until any existing write stall is cleared, then sets `writes_blocked_ = true` *before* draining the writer queue. Setting the flag first matters: the drain releases `mutex_` inside `Wait()`, so without the gate-first ordering a steady stream of new writers could keep enqueueing and starve `DeleteRange`. The flag check at the top of `WriteInternal` (before `push_back`) means once the flag is set, no new entries land in `writers_`.
2. The range-delete batch itself is committed through `WriteInternal(..., true)`,

where the `bypass_stall` flag allows this operation to proceed while the global stall is active.

3. On completion, it clears `writes_blocked_` and broadcasts `SignalAll()` so blocked writers resume.

This gives `DeleteRange` a predictable critical section with respect to foreground writes, while preserving LevelDB's existing background-compaction concurrency.

3.2.3 Efficiency note

The current implementation has cost proportional to the number of keys in the range, since it scans every key and issues one tombstone per key. A more advanced approach (like range tombstones in RocksDB's `DeleteRange`) would encode the deletion as a single record instead of $O(n)$ tombstones. For the scope of this assignment the key-by-key approach is appropriate and correct.

If the iterator returns a non-OK status during the scan, we skip the write and propagate the error to prevent partial state from accumulating.

4 Manual Full Compaction: `ForceFullCompaction()`

4.1 API Definition

Added to `include/leveldb/db.h`:

```
1 // Will use the above described method to compact the entire db
2 virtual Status ForceFullCompaction() = 0;
```

Listing 6: `ForceFullCompaction` API in `include/leveldb/db.h`

The override is declared in `db/db_impl.h`.

4.2 Compaction Statistics and Concurrency Fields

To track what the compaction actually did, we introduced a `FullCompactionStats` struct in `db/db_impl.h`:

```
1 struct FullCompactionStats {
2     int num_compactions;
```

```

3  int num_input_files;
4  int num_output_files;
5  uint64_t bytes_read;
6  uint64_t bytes_written;
7
8  FullCompactionStats()
9      : num_compactions(0), num_input_files(0),
10        num_output_files(0), bytes_read(0), bytes_written(0) {}
11 };

```

Listing 7: FullCompactionStats in db/db_impl.h

The latest implementation stores an active stats pointer and a write-stall flag in DBImpl:

```

1  FullCompactionStats* full_compaction_stats_ GUARDED_BY(mutex_);
2  std::atomic<bool> writes_blocked_ GUARDED_BY(mutex_);
3  std::atomic<bool> reads_blocked_ GUARDED_BY(mutex_);
4
5  Status WriteInternal(const WriteOptions& options,
6                      WriteBatch* updates,
7                      bool bypass_stall);

```

Listing 8: Concurrency/stat members in db/db_impl.h

In addition, each queued writer now carries a `bypass_stall` bit (in `db/db_impl.cc`) so selected internal writes can proceed even while external writes are intentionally stalled.

4.3 ForceFullCompaction Implementation

The top-level implementation is in `db/db_impl.cc`:

```

1  Status DBImpl::ForceFullCompaction() {
2      FullCompactionStats stats{};
3      {
4          MutexLock l(&mutex_);
5          while (writes_blocked_.load(std::memory_order_acquire)) {
6              background_work_finished_signal_.Wait();
7          }
8          // Gate new writers/readers BEFORE draining: otherwise the drain
9          releases
10         // mutex_ in Wait() and fresh callers can keep adding to writers_,

```

```

10 // starving FFC.
11 writes_blocked_.store(true, std::memory_order_release);
12 reads_blocked_.store(true, std::memory_order_release);
13 while (!writers_.empty()) background_work_finished_signal_.Wait();
14 // Wait for any in-flight background compaction to finish so its
    stats
15 // are not attributed to this FFC.
16 while (background_compaction_scheduled_) {
17     background_work_finished_signal_.Wait();
18 }
19 full_compaction_stats_ = &stats;
20 }
21
22 CompactRange(nullptr, nullptr);
23
24 printf("----- Manual Compaction Statistics -----\n");
25 printf("Compactions executed: %d\n", stats.num_compactions);
26 printf("Input files:           %d\n", stats.num_input_files);
27 printf("Output files:           %d\n", stats.num_output_files);
28 printf("Total bytes read:       %llu\n", (unsigned long long)stats.
    bytes_read);
29 printf("Total bytes written:   %llu\n",
30     (unsigned long long)stats.bytes_written);
31 printf("-----\n");
32
33 {
34     MutexLock l(&mutex_);
35     full_compaction_stats_ = nullptr;
36     reads_blocked_.store(false, std::memory_order_release);
37     writes_blocked_.store(false, std::memory_order_release);
38     background_work_finished_signal_.SignalAll();
39 }
40
41 return Status::OK();
42 }

```

Listing 9: ForceFullCompaction in db/db_impl.cc

The critical concurrency decision here is to stall foreground writes for the full-compaction window. This avoids races where new writes could enter MemTable/WAL while compaction is trying to produce a globally compacted view.

To block all the reads during Full Compaction a new flag `reads_blocked_` was

added and a stall based on the flag was added to `Get` and `NewInternalIterator` functions. The flag is set and unset by the `ForceFullCompaction` function along with the `writes_blocked_` flag.

Two ordering details inside the entry critical section are load-bearing:

- **Gate before drain.** Both stall flags are set *before* draining `writers_`. The drain calls `Wait()` which releases `mutex_`, so if the flags were set after the drain, fresh callers admitted during those wait-windows could keep enqueueing and starve FFC indefinitely. Setting the flags first means `WriteInternal/Get/NewInternalIterator` stall before `push_back`, so the queue can only shrink.
- **Drain pre-existing background compaction.** After the writer drain we also wait on `background_compaction_scheduled_`, otherwise an in-flight automatic compaction's stats would be wrongly attributed to this FFC once `full_compaction_stats_` is published. We do *not* need a symmetric drain at exit: `TEST_CompactRange` (called by `CompactRange`) already drains `background_compaction_scheduled_` before returning, so by the time `CompactRange` exits no background work is outstanding.

```

1 while (reads_blocked_.load(std::memory_order_acquire)) {
2     background_work_finished_signal_.Wait();
3 }

```

Listing 10: Added to `Get`, `NewInternalIterator` in `db/db_impl.cc`

4.4 Deadlock Avoidance in `TEST_CompactMemTable`

The most recent push fixes a self-deadlock: `CompactRange()` calls `TEST_CompactMemTable()`, and under `ForceFullCompaction` the write-stall flag is already set. If `TEST_CompactMemTable()` used normal `Write()`, it could block behind its own stall.

```

1 Status DBImpl::TEST_CompactMemTable() {
2     Status s = WriteInternal(WriteOptions(), nullptr,
3                             /*bypass_stall=*/true);
4     ...
5 }

```

Listing 11: Bypass path used by `TEST_CompactMemTable`

`WriteInternal` also enforces an important queueing rule: non-bypass writers wait *before* entering `writers_`. This prevents head-of-line blocking where a stalled non-bypass writer could otherwise sit ahead of an internal bypass writer.

4.5 Metric Collection in the Compaction Worker

Two places in `db_impl.cc` now accumulate into `full_compaction_stats_` when it is non-null:

In `WriteLevel0Table`. MemTable flush output contributes bytes/output-file counters:

```
1 if (FullCompactionStats* fc_stats_ = full_compaction_stats_) {  
2     fc_stats_->bytes_written += meta.file_size;  
3     fc_stats_->num_compactions++;  
4     fc_stats_->num_output_files++;  
5 }
```

In `DoCompactionWork`. Compaction reads/writes and file counts are added per compaction unit:

```
1 if (FullCompactionStats* fc_stats_ = full_compaction_stats_) {  
2     fc_stats_->bytes_read += stats.bytes_read;  
3     fc_stats_->bytes_written += stats.bytes_written;  
4     fc_stats_->num_compactions += 1;  
5     fc_stats_->num_input_files += compact->compaction->num_input_files  
6         (0);  
7     fc_stats_->num_input_files += compact->compaction->num_input_files  
8         (1);  
9     fc_stats_->num_output_files +=  
10         compact->outputs.size();  
11 }
```

Listing 12: Stats accumulation in `DoCompactionWork` (`db/db_impl.cc`)

The `stats.bytes_read` and `stats.bytes_written` values are taken from existing internal `CompactionStats`; the new code only routes them into the externally reported aggregate.

5 Summary of File Changes

The vast majority of the logic sits in `db/db_impl.cc`, as expected - the public headers only needed declarations, and the implementation files did all the heavy lifting.

Table 1: Overview of modified files and change counts

File	Purpose	Insertions	Deletions
include/leveldb/db.h	Public API declarations	11	0
db/db_impl.h	Class declarations + structs	35	3
db/db_impl.cc	All implementation logic	147	11
db/write_batch.cc	Include formatting only	2	0
Total		195	14

6 End-to-end Sanity Test

To validate that the three new APIs (`Scan`, `DeleteRange`, `ForceFullCompaction`) compose correctly on a fresh database, we wrote a small driver that exercises each one in turn and cross-checks the results with `Get`.

6.1 What the test does

1. Open a fresh DB at `/tmp/testdb`, insert 200 keys `key00000` – `key00199` with values `val00000` – `val00199`.
2. `Scan("key00050", "key00100")` – expects exactly 50 results in sorted order.
3. `Scan("key00100", "key00050")` (reversed range) – expects 0 results and `Status::OK`.
4. `DeleteRange("key00050", "key00150")` – expects 100 keys gone afterwards.
5. `Get` every key in `[0, 200)` and count present/missing – expects 100/100.
6. `ForceFullCompaction()` – prints the compaction statistics.
7. Re-count present/missing – expects 100/100 (compaction must not change visible state).

6.2 Output

```

1 ----- Manual Compaction Statistics -----
2 Compactions executed: 1
3 Input files:          0
4 Output files:         1
5 Total bytes read:     0

```

```
6 Total bytes written: 5462
```

```
7 -----
```

Listing 13: ForceFullCompaction stats on a fresh DB

6.3 Reading the FFC numbers

The dataset is small enough (200 puts + 100 tombstones) to fit entirely in the MemTable, so when FFC runs against a fresh DB there are no on-disk SSTables to merge. The single counted “compaction” is the MemTable flush issued from `TEST_CompactMemTable`: it produces one new L0 file of 5462 bytes and reads zero input files (the source is the in-memory MemTable, not an SSTable). The subsequent `TEST_CompactRange` calls in `CompactRange` have no level with overlapping files to operate on, so they short-circuit. On a DB that already has data spread across multiple levels, the same call would report several compactions with non-zero input/output file counts.

All other assertions in the test (`Scan` count, reversed-range emptiness, post-`DeleteRange` 100/100, post-FFC 100/100) also held, confirming that FFC is value-preserving.